useReducer Hook

- The useReducer is an alternative to the useState hook for managing state in functional components. The useReducer hook is better suited for managing complex state logic while useState is best for simple state changes.
- When the state logic becomes too complicated or when you need to handle state changes in a more predictable and manageable way, the useReducer hook is your best bet.

Syntax

```
const [state, dispatch] = useReducer(reducerFunction, initialState);
```

- A useReducer is a hook in React that allows you add a reducer to your component.
- It takes in the reducer function and an initialState as arguments.
- The useReducer also returns an array of the current state and a dispatch function.
    - state: represents the current value and is set to the initialState value during the initial render.
    - dispatch: is a function that updates the state value and always triggers a re-render, just like the updater function in useState.
    - reducer: is a function that houses all the logic of how the state gets updated. It takes state and action as arguments and returns the next state.
    - initialState: houses the initial value and can be of any type.

reducer function

- This function determines how the state gets updated and contains all the logic through which the next state will be calculated.
- Basically, reducers house the logic that is usually placed inside of an event handler when using useState.
- This makes it easier to read and debug when you are not getting desired results.
- The reducer function is always declared outside of your component and takes in a current state and action as arguments.

  ```
  function reducer(state, action) {
  }
  ```
- An action is an object that typically has a type property which identifies a specific action.
- Actions describe what happens and contains information necessary for the reducer to update the state.

- Conditional statements are used to check the action types and perform a specified operation that would return a new state value.
- Conditional statements like if and switch can be used in reducers.

Dispatch Function

- This is a function returned by the useReducer hook and is responsible for updating state to a new value.
- The dispatch function takes the action as its only argument.
- We can place the dispatch function inside an event handler function.
- Remember, actions come with a type property so we have to specify when we call the dispatch function.

Example1: Counter App

```
import React from 'react'
import { useReducer } from 'react'

const reducer = (state, action)=>{
  switch(action.type){
     case "INCREMENT":
        return{count: state.count + 1};
     case "DECREMENT":
        return{count: state.count - 1};
     default:
        return state;
  }
}
function CounterComponent() {
  const [state, dispatch] = useReducer(reducer, {count:0})
 return (
  <div>
     <p>Count: {state.count}</p>
```

```
      <button onClick={()=>dispatch({type: "INCREMENT"})}>Increase
Count</button>
      <button onClick={()=>dispatch({type: "DECREMENT"})}>Decrease
Count</button>

  </div>
 )
}

export default CounterComponent
```

- The useReducer hook is initialized with a reducer function and an initial state object containing count.
- The reducer function handles state transitions based on the dispatched action types, making the state management more structured and scalable.
- The Counter component displays the current count and provides two buttons that dispatch actions to either increment or decrement the count value.
- The advantage of using useReducer in this scenario is that it provides a centralized approach to handling state updates, making it easier to manage complex state logic in larger applications.
- Additionally, by encapsulating the state logic in a separate function, the component remains clean and easier to maintain.

Example 2: A shopping Cart where one can add and remove items

```
import React from 'react'
import { useReducer } from 'react';

function reducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return { items: [...state.items, action.payload] };
```

```
      case 'REMOVE_ITEM':
        return { items: state.items.filter((item, index) => index !== action.payload) };
      default:
        return state;
    }
  }

  function Cart() {
    const [state, dispatch] = useReducer(reducer, { items: [] })

    return (
      <div>
        <h1>Shopping Cart</h1>
        <button onClick={() => dispatch({ type: "ADD_ITEM", payload: 'Item' +
(state.items.length + 1) })}>Add to Cart</button>
        <ul>
          {state.items.map((item, index) => (
            <li key={index}>{item} <button onClick={() => dispatch({ type:
"REMOVE_ITEM", payload: index })}>Remove</button></li>
          ))}
        </ul>
      </div>
    )
  }

  export default Cart
```

The reducer function takes two parameters:

- state: Represents the current state of the cart.

- action: An object that contains a type and optionally a payload, which tells the reducer how to modify the state.

The switch statement inside the reducer handles two types of actions:

1. Adding an Item (ADD_ITEM)
    - The new item (passed as payload) is appended to the items array using the spread operator (...).
    - The updated array is then returned as the new state.
2. Removing an Item (REMOVE_ITEM)
    - The item is removed by filtering out the item at the specified index (payload).
    - A new array without the removed item is returned.

If the action type doesn't match any known cases, the reducer simply returns the current state to avoid unwanted modifications.

The Cart component is the main UI that interacts with the reducer. It consists of:

- State and Dispatch Setup

  The useReducer hook initializes the state with an empty array of items and provides a dispatch function to modify the state.
- Adding an Item

  Clicking the "Add to Cart" button triggers dispatch({ type: "ADD_ITEM", payload: ... }), which:
    - Generates a new item label ("Item X"), where X is the next index.
    - Sends this as the payload in the ADD_ITEM action.
- Rendering the List of Items

  The ul tag dynamically displays all items in the cart. Each item is mapped into an li element.
- Removing an Item

  Each li contains a "Remove" button. Clicking it triggers dispatch({ type: "REMOVE_ITEM", payload: index }), where index is the position of the item in the array. The reducer then filters out this item.

For more practice -
https://medium.com/zestgeek/mastering-reacts-usereducer-hook-a-comprehensive-guide-with-examples-e48c2f306566